PhoneMail

Complete System Documentation

Email Application Using Phone Numbers as Email IDs

Generated: September 24, 2026

Table of Contents

1. Overview & Purpose

2. Prerequisites & Inputs

3. Step-by-Step Operational Flow

4. Decision Points & Branching Logic

5. Outputs & End States

6. Integration Points

7. Common Pitfalls & Troubleshooting

8. Scalability Considerations

9. Variations & Context-Dependent Flows

1. Overview & Purpose

PhoneMail is an email application that uses phone numbers as email identifiers (e.g., 9876543210@phonemail.com). Its core purpose is to democratize email access for users in regions with high mobile penetration but limited traditional email adoption. It replaces complex email workflows with familiar chat-style conversations (inspired by WhatsApp/Spike Mail) and enables account creation via voice (IVR) or SMS, making it accessible without internet-enabled devices for initial signup.

Why it matters:

- Bridges the gap between email and messaging apps
- Enables offline signup via toll-free voice calls
- Reduces friction for users unfamiliar with traditional email
- Organizes emails as conversations (familiar UX from messaging apps)
- Supports users without smartphones initially (IVR + SMS)

2. Prerequisites & Inputs

Infrastructure Requirements

• Backend server (Node.js/Express)

• Database (PostgreSQL/MongoDB)

• Email service provider (SMTP/nodemailer)

• SMS/IVR gateway (Twilio or similar)

• OTP provider (Twilio, Firebase, or custom)

• Mobile development environment (React Native/Flutter)

• Web development stack (React + TypeScript)

• Docker & docker-compose

• Redis (optional, for session caching)

External Services Needed

• Twilio Account (toll-free number, SMS, OTP)

• Email server (for sending/receiving emails)

• App store accounts (for mobile deployment)

User-Side Inputs

• Phone number (for account creation & login)

• OTP (received via SMS or auto-detected)

• Password (fallback if OTP unavailable)

• Terms & Conditions acceptance (mobile)

• Device permissions (SMS, contacts, phone state)

System Inputs

• Email metadata (sender, recipient, subject, body, attachments)

• Conversation grouping rules (emails from same sender)

• Filter criteria (All, Unread, Attachments, Favorites)

3. Step-by-Step Operational Flow

3.1 Account Creation

Path A: Via Toll-Free Voice (IVR)
Step 1: User dials toll-free number → Twilio receives incoming call → IVR engine plays prompt

Step 2: User presses '1' → IVR captures input → Prompt asks for 10-digit phone number

Step 3: Phone number captured via speech/keypad → Backend validates format

Step 4: Verification SMS sent with 6-digit OTP → Backend generates OTP → Twilio sends SMS

Step 5: User enters OTP → Validated against stored OTP → Account created or retry (max 3x)

Path B: Via Web Portal (Registration Only)
Step 1: User visits web portal → Registration page with 2-field form loads

Step 2: User enters phone number → Format validation (10 digits) → Backend generates OTP

Step 3: OTP sent via SMS → User enters OTP (auto-detected or manual)

Step 4: OTP validated → Account created → Both fields reset for next user

Path C: Via Mobile Client
Screen 1: Language selection (English, Hindi, regional languages)

Screen 2: Terms & Conditions (scrollable, accept checkbox)

Screen 3: Phone number auto-detection → User can edit → Permissions requested

Screen 4: OTP auto-detected from SMS → Auto-populated → User validates → Inbox access

3.2 Sending an Email

Users can send emails in two modes: **Chat-style** (search recipient → type message) or **Traditional** (full email form with To, CC, BCC, Subject, Attachments). Group emails (2+ recipients) create a new group conversation. Replies to emails are linked to original messages with swipe-right action for traditional view. Each message can be replied to only once. Once sent, emails are marked in the database, user sees ✓✓ confirmation.

3.3 Receiving & Accessing Emails

Incoming emails are parsed by the backend (IMAP polling or webhook), stored in the database, and grouped into conversations by sender. Recipients are notified via push notification (if mobile app is active) or SMS (if no app). Inbox display organizes emails as conversations (mobile) or email list (web). Search functionality allows filtering by sender, subject, date. Filter chips (All, Unread, Attachments, Favorites) narrow results.

3.4 SMS Notifications

Triggered for users without mobile app (registered via IVR, web portal, or web client). Format: 'You have received an email from [Sender Name/Phone]. Subject: [Subject].' Sent via Twilio trial account (international number). Delivery tracked (sent, delivered, failed). Throttled to max 1 SMS per email per hour to prevent spam.

4. Decision Points & Branching Logic

Decision Point	Condition	True Path	False Path
Account Method?	User calls toll-free	IVR flow	Check web portal
OTP Available?	Free OTP provider accessible	Send OTP via SMS	Use password auth
OTP Validation	OTP matches & not expired	Create account	Retry (max 3x)
Has Mobile App?	User has app installed & logged in	Push notification	SMS notification
Recipient Valid?	Phone number exists	Send email	Error message
Group Email?	Recipients count > 1	Create group conversation	1-to-1 conversation

5. Outputs & End States

Successful Completion States

Account Creation Success: User created in database, Email ID: [phone]@phonemail.com, Status: 'active', User can log in & receive emails

Email Sent Success: Email stored (status: 'sent'), Recipient notification sent, Sender sees ✓✓ indicator

Email Received Success: Email stored & indexed, Notification delivered, Visible in recipient's inbox/conversation

Search Completed: Results displayed (ranked by relevance), User can tap to open conversation

Failure/Exception States

Invalid phone number: Error message → User retries input

OTP expired: Error message → Send new OTP

OTP max retries exceeded: Rate limit → Retry after 15 mins

Recipient doesn't exist: Error message → Suggest similar numbers or create new account

SMTP failure: Email queued for retry (exponential backoff)

SMS delivery failed: Log to database, retry on next email

Database connection lost: Automatic reconnect with exponential backoff

IVR timeout: Replay prompt (max 3x) then disconnect

6. Integration Points

Upstream Integrations (External Systems → PhoneMail)

Email Providers: IMAP/POP3 server or webhook → Email metadata → Database → User inbox

Twilio (SMS/IVR): Incoming call webhook (IVR) → Backend → Account creation; Backend → SMS → User phone

OTP Providers: Backend → OTP service → SMS to user → User input → Backend validation

Authentication Systems: Device OS (SIM/phone number detection), SMS app (OTP auto-detection), Contact list (recipient suggestions)

Downstream Integrations (PhoneMail → External Systems)

Email Servers (SMTP): PhoneMail backend → SMTP server → Recipient inbox

App Stores: Mobile app binary → App Store & Google Play → User device

Analytics & Logging: App/backend events → Analytics service (Mixpanel, Sentry)

Push Notification Services: Backend → FCM/APNs → User mobile device

7. Common Pitfalls & Troubleshooting

OTP Delivery Failures: SMS doesn't arrive within 2 mins. Root cause: Twilio throttling, network delays. Fix: Implement retry logic, show countdown timer, provide password fallback.

Email Grouping Errors: Emails from same sender in multiple conversations. Root cause: Sender phone variations (+91, 0, country code). Fix: Normalize phone numbers on storage.

SMS Notification Spam: 10+ SMS for same email. Root cause: No deduplication. Fix: Track notification timestamp, throttle (max 1 SMS/email/hour).

IVR Timeouts: User disconnected after 30s. Root cause: Network latency, user hesitation. Fix: Extend timeout to 60s, replay prompt after 15s of silence.

Mobile App Crashes: 5000+ emails cause freezing. Root cause: Loading all emails at once. Fix: Implement lazy loading, pagination (20 conversations initially).

Database Pool Exhaustion: Backend stops responding after 1000 concurrent users. Root cause: Pool size too small. Fix: Scale to 30-50, add query timeouts (5s).

SMTP Rate Limiting: Emails fail after 100/min. Root cause: Email provider limits. Fix: Implement message queue, send with backoff.

Web Portal CSRF: Attacker redirects user to malicious site. Root cause: No CSRF token. Fix: Add csrf-token to forms, use SameSite cookies.

8. Scalability Considerations

At 1,000 Users (MVP)

- Single Node.js server, single PostgreSQL instance
- In-memory Redis for OTP caching (2GB RAM)
- No CDN needed
- Twilio sandbox for SMS testing
- Mobile app limited to iOS/Android native builds

At 10,000 Users (Growth Phase)

- Load balancer (NGINX) → 2-3 Node.js instances
- PostgreSQL with read replicas
- Redis cluster for sessions + cache (10GB)
- Message queue (Redis Streams) for email sending
- CDN for static assets (web client)
- Separate webhook server for incoming emails
- Mobile app released on app stores

At 100,000+ Users (Scale)

- Kubernetes cluster (2-3 nodes minimum)
- PostgreSQL sharding by phone number prefix
- Redis cluster + Memcached hybrid
- Dedicated email delivery service (SendGrid, AWS SES)
- Message broker: RabbitMQ or Kafka for email queuing
- Elasticsearch for full-text search
- S3 for attachment storage
- Analytics service (Mixpanel, Segment)
- CDN with edge caching
- SMS gateway redundancy (Twilio + Vonage)

9. Variations & Context-Dependent Flows

Context	Variation
Offline-first user	Signup via IVR only, emails checked on web portal once internet available, SMS read aloud via accessibil
No OTP provider	Default to password auth with minimal complexity (6-digit numeric PIN alternative)
Group company deployment	Custom domain: [phone]@company.phonemail.com, admin controls, SSO integration
Regional compliance (GDPR/CCPA)	Data residency in specific region, deletion on request, consent on signup, audit logs
Accessibility (blind/deaf user)	Text-to-speech for IVR, SMS via email-to-SMS gateway, high-contrast web UI

Critical Success Factors

• **OTP is the bottleneck** — Invest in reliable SMS delivery, provide password fallback

• **Conversation grouping logic** — Get phone number normalization right from day 1

• **Push vs SMS decision tree** — Correctly detect mobile app presence, avoid notification spam

• **Attachment handling** — Support files up to 25MB, handle MIME encoding properly

• **Onboarding UX** — WhatsApp familiarity wins user adoption; every friction point hurts conversion

• **IVR reliability** — Toll-free number must be bulletproof; failed IVR = lost user

• **Database indexing** — Conversations and emails tables need indices on (user_id, sender_id), (created_at)

• **Monitoring & alerts** — Track OTP failure rate, email delivery latency, SMS cost per user, crash rates

---